Rapport de Projet de Maitrise

Sujet :

Systeme d'Intelligence Collective pour la Classification Medicale en Environnement IoT Distribue

Realise par :

Ayoub Mrani Alaoui
Jihane El Aouni
Douae Chater
Najlae Sebbar

Encadre par :

Mr Said Ohamouddou

Resume Executif

Ce projet developpe un systeme d'intelligence collective pour la classification d'images medicales dans un environnement IoT distribue. Le systeme combine trois machines virtuelles configurees avec des modeles optimises (quantification et elagage) executant des inferences paralleles, aggregees par un vote pondere central.

Resultats Cles:

- **Precision Collective:** 100% sur 10 images de test
- **Compression Optimale:** Q5 atteint 48.09% de reduction (8.51 MB vers 4.42 MB)
- **Vitesse de Pointe:** P3 offre 33.54 ms/image (6.1x plus rapide)
- **Consensus Unanime:** 100% d'accord entre trois VMs
- **Integration IoT:** 40 points de telemetrie publies avec succes
- **Systeme d'Alarmes:** 30+ alarmes validees et declenchees

Contents

Resume Executif **1**

1 Introduction et Contexte **4**
1.1 Motivation du Projet 4
1.2 Architecture Proposee 4

2 Phase 1: Entrainement du Modele de Base **5**
2.1 Configuration 5
2.2 Resultats 5

3 Phase 2: Optimisation par Quantification et Elagage **7**
3.1 Techniques de Quantification (Q1-Q5) 7
3.2 Techniques d'Elagage (P1-P3) 7

4 Phase 3: Evaluation Systematique Multi-VM **9**
4.1 Profils Materiels 9
4.2 Matrice de Precision 9
4.3 Matrice Temps Inference 10

5 Phase 4: Selection Optimale et Ponderation **11**
5.1 Scoring Pondere 11
5.2 Scores et Selection 11
5.3 Poids de Vote Pondere 11

6 Phase 5: Intelligence Collective Distribuee **12**
6.1 Architecture Systeme 12
6.2 Mecanisme de Vote Pondere 12
6.3 Resultats Phase 5 12

7 Phase 6: Integration MQTT IoT Complete **14**
7.1 Infrastructure MQTT 14
7.2 Telemetrie en Temps Reel 14
7.3 Systeme d'Alarmes 14
7.4 Resultats Test Alarmes 15

8 Defis Rencontres et Solutions **16**
8.1 Defi 1: Etat Dictionnaire Inconsistant 16
8.2 Defi 2: Consommation RAM Critique 16
8.3 Defi 3: Commutation MQTT Credentials 16
8.4 Defi 4: Calcul Taux de Consensus 16

9 Conclusion **17**
9.1 Recapitulatif 17
9.2 Performances Finales 17
9.3 Contributions 17

A Screenshots **18**
A.1 Machines Virtuelles avec MQTT Connectees 18
A.2 Dashboard ThingsBoard avec Alarmes en Temps Reel 19

B Fichiers Generes **20**

Chapter 1

Introduction et Contexte

1.1 Motivation du Projet

La classification d'images medicales constitue une tache critique en diagnostic assiste par ordinateur. Les defis majeurs incluent:

1. **Precision:** Necessaire une exactitude proche de 100% pour applications medicales
2. **Latence:** Systemes edge/IoT imposent contraintes strictes ($<$ 300 ms)
3. **Ressources:** Limitation drastique RAM et CPU (500 MB typiquement)
4. **Scalabilite:** Support de multiples materiels heterogenes
5. **Robustesse:** Surveillance en temps reel et alarmes

1.2 Architecture Proposee

Notre approche combine:

- **Optimisation:** 8 techniques (quantification, elagage) testees
- **Distribution:** 3 machines virtuelles en parallele
- **Agregation:** Vote pondere base sur performance
- **Telemetrie IoT:** Integration MQTT ThingsBoard
- **Systeme d'Alarmes:** Detection anomalies temps reel

Chapter 2

Phase 1: Entrainement du Modele de Base

2.1 Configuration

Parametre	Valeur
Architecture	MobileNetV2
Dataset	LC25000 (Cancer Colon+Poumon)
Classes	5
Epochs	5
Batch Size	32

Table 2.1: Configuration Phase 1

2.2 Resultats

Le modele de base MobileNetV2, entraine sur le dataset LC25000 comprenant des images de cancers du colon et du poumon, delibre des performances exceptionnelles. Apres 5 epochs d'entrainement sur GPU CUDA avec une taille de batch de 32, le modele atteint une precision de validation de 99.9%, confirmee par une precision de test de 99.88%.

Cette performance de base est remarquable car elle etablit une reference solide pour evaluer l'impact des techniques d'optimisation qui seront appliquees en Phase 2. Le modele, bien que performant, occupait 8.51 MB de memoire, ce qui pose des defis pour le deploiement sur des dispositifs constraints comme les IoT.

Le temps d'inference moyen de 0.1639 ms par image sur GPU montre que le modele baseline est rapidement execute, mais nous devons verifier comment cette latence evolue sur des processeurs CPU moins puissants typiques des environnements edge et IoT.

Metrique	**Valeur**
Precision Test	99.88%
Rappel	99.88%
Score F1	99.88%
Temps Entrainement	7.19 minutes
Taille Modele	8.51 MB
Temps Inference	0.1639 ms/image

Table 2.2: Resultats Phase 1 - Performance Baseline

Ces metriques etablissent notre point de reference pour les comparaisons futures. Chaque technique d'optimisation sera evaluee non seulement sur sa capacite a reduire la taille ou accelerer l'inference, mais aussi sur sa capacite a preserver ou ameliorer cette precision exceptionnelle.

Chapter 3

Phase 2: Optimisation par Quantification et Elagage

3.1 Techniques de Quantification (Q1-Q5)

La quantification represente une strategie majeure pour reduire la taille des modeles tout en maintenant precisions. En reduisant la precision numerique (float32 vers int8, float16), on economise de la memoire et on accelere les operations.

Nous avons teste cinq techniques de quantification differentes, chacune avec ses propres caracteristiques et compromis. La technique Q5 (Precision Mixte FP16) s'est revelee particulierement efficace, accomplissant une reduction de taille de 48.09% (8.51 MB vers 4.42 MB) tout en preservant la precision de 99.88%.

Cette reduction majeure de taille rend le modele eligible pour des deploiements avec bande passante limitee, ce qui est crucial pour les architectures IoT distribuees ou les mises a jour de modeles doivent transiter sur des connexions constraintes.

ID	Technique	Taille	Precision	Grade
Q1	Dynamic Quant	8.72 MB	99.88%	A
Q2	PTQ Statique	8.72 MB	99.88%	A
Q3	QAT	8.72 MB	99.88%	A
Q4	Poids Seulement	8.72 MB	99.88%	A
lightblue Q5	lightblue Mixte FP16	lightblue 4.42 MB	lightblue 99.88%	lightblue A+
P1	Elagage NS	8.74 MB	32.17%	C
P2	Elagage Struct	8.74 MB	31.45%	C
lightblue P3	lightblue Elagage Magn	lightblue 8.74 MB	lightblue 99.92%	lightblue A

Table 3.1: Resultats Phase 2 - Quantification et Elagage

3.2 Techniques d'Elagage (P1-P3)

L'elagage complementent la quantification en supprimant les poids peu importants du modele. Nous avons teste trois variantes d'elagage, avec resultats tres differents.

Les techniques d'elagage non-structuree (P1) et structuree (P2) ont malheureusement sacrifie beaucoup de precision pour gagner un peu de vitesse, les rendant inadaptes pour les applications medicales critiques. Cependant, P3 (Elagage par Magnitude) a obtenu des resultats remarquables: non seulement elle a preserve 99.92% de precision (meme

legerement mieux que baseline), mais elle a egalement delivre des temps d'inference ultra-rapides de 0.1915 ms.

Conclusions Phase 2:

En resume, Phase 2 a identifie deux candidats exceptionnels pour le deploiement:

- **Q5 (Precision Mixte FP16):** Meilleur pour les scenarios de bande passante limitee avec reduction de taille de 48%
- **P3 (Elagage Magnitude):** Parfait pour les scenarios latence-critique avec performance exceptionnelle

Chapter 4

Phase 3: Evaluation Systematique Multi-VM

4.1 Profils Materiels

Avant d'evaluer les modeles, nous avons configure trois machines virtuelles avec des profils materiels distincts pour simuler un ecosysteme IoT/Edge/Production realiste:

VM	**RAM**	**CPU**	**Role**
VM1	423 MB	1 Core	IoT Contraint
VM2	926 MB	2 Cores	Edge Server
VM3	1933 MB	2 Cores	Production

Table 4.1: Profiles Materiels VMs - Ecosysteme Heterogene

VM1 simule un dispositif IoT extreme, avec seulement 423 MB de RAM et un processeur monocore. VM2 represente un serveur edge intermediaire avec ressources moderees. VM3 simule un environnement production avec ressources abondantes. Cette diversite nous permet de valider que nos modeles optimises fonctionnent correctement sur differentes configurations.

4.2 Matrice de Precision

La matrice de precision 3x8 reveale des resultats tres importants. Seuls deux modeles - P3 et Q5 - ont maintenu une precision parfaite de 100% sur toutes les machines virtuelles:

VM	Q1	Q2	Q3	Q4	Q5	P1	P2	P3
VM1	1.25%	11.25%	0%	92.5%	lightblue100%	45.62%	51.88%	100%
VM2	99.38%	51.88%	0%	0%	lightblue100%	45.62%	51.88%	100%
VM3	0%	90%	0%	0%	lightblue100%	45.62%	51.88%	100%

Table 4.2: Matrice Precision Phase 3 - 3 VMs x 8 Modeles

Cette decouverte est cruciale: Q1, Q2, Q3 et Q4 montrent une precision tres variable selon la VM, indiquant des problemes de compatibilite ou de corruption des poids lors de la serialisation. Seuls P3 et Q5 sont robustes et pretes pour la production.

4.3 Matrice Temps Inference

Au-dela de la precision, les performances de latence varient significativement selon le modele et le materiel:

VM	Q1	Q2	Q3	Q4	Q5	P1	P2	P3
VM1	174	184	229	230	234	188	198	204
VM2	54.5	52	47.4	45.2	lightblue43	90.2	92.1	79.6
VM3	34.9	33.9	37.2	43	56.7	50.2	33.2	lightblue33.5

Table 4.3: Matrice Temps Inference Phase 3 (ms/image)

L'observation majeure est que P3 offre une acceleration spectaculaire de 6.1x en passant de VM1 (204ms) a VM3 (33.5ms), tout en conservant 100% de precision. Ceci demontre que l'elagage par magnitude est particulierement efficace sur les materiels modernes multi-coeurs.

Chapter 5

Phase 4: Selection Optimale et Ponderation

5.1 Scoring Pondere

Chaque VM a des criteres de ponderation specifiques:

VM1 (IoT) RAM 40% + CPU 40% + Precision 20%

VM2 (Edge) RAM 30% + Vitesse 30% + Precision 40%

VM3 (Production) Precision 60% + Vitesse 25% + RAM 15%

5.2 Scores et Selection

VM	Modele	Score Phase 4
lightblue VM1	lightblue Q5	lightblue 0.68
lightblue VM2	lightblue Q5	lightblue 0.96
lightblue VM3	lightblue P3	lightblue 0.97

Table 5.1: Selection et Scores Phase 4

5.3 Poids de Vote Pondere

VM	Poids Brut	Poids Normalise
VM1	0.68	26.05%
VM2	0.96	36.78%
VM3	0.97	37.16%
Total	**2.61**	**100.00%**

Table 5.2: Poids de Vote Pondere

Chapter 6

Phase 5: Intelligence Collective Distribuee

6.1 Architecture Systeme

Composant	Adresse	Modele	Poids
VM1	192.168.52.10:5001	Q5 (FP16)	26.05%
VM2	192.168.52.20:5002	Q5 (FP16)	36.78%
VM3	192.168.52.30:5003	P3 (Elagage)	37.16%

Table 6.1: Architecture Phase 5

6.2 Mecanisme de Vote Pondere

L'intelligence collective repose sur un mecanisme de vote pondere qui combine les predictions de chaque VM. Au lieu de prendre simplement la majorite, chaque VM se voit assigner un poids base sur sa performance en Phase 4.

Le vote pondere combine predictions:

$$\text{Score}_{classe} = \sum_{i=1}^{3} (\text{Confiance}_{i,classe} \times w_i)$$

Ou w_i sont les poids normalises: 26.05% pour VM1, 36.78% pour VM2, et 37.16% pour VM3. Ceci signifie que VM3, avec sa performance optimale en Phase 4, exerce une influence legerement plus grande sur la decision collective.

6.3 Resultats Phase 5

L'execution du systeme d'intelligence collective sur 10 images de test medicales a produit des resultats extraordinaires. Chaque VM a execute independamment son modele optimise, et l'orchestrateur a agrege les resultats via le vote pondere.

Metrique	**Valeur**
Nombre Images	10
Precision Collective	100% (10/10)
Confiance Moyenne	99.98%
Votes Unanimes	10/10 (100%)
Tentatives Augmentation	0
VMs Saines	3/3

Table 6.2: Resultats Phase 5 - Intelligence Collective Distribuee

La precision collective de 100% signifie que chaque diagnostic produit par le systeme etait correct. Encore plus remarquable, les trois VMs ont ete unanimes sur toutes les predictions, ce qui indique un consensus fort et une haute confiance dans les diagnostics.

La confiance moyenne de 99.98% est exceptionnelle pour une application medicale. Cela signifie que le systeme n'a jamais doute de ses decisions. En fait, le systeme de re-evaluation avec augmentation de donnees (declenchee si confiance $< 70\%$) n'a jamais ete active, car toutes les predictions ont depasse le seuil de confiance.

Les VMs individuels ont egalement maintenu 100% de precision:

- VM1 (Q5): 100% (10/10 images correctes)
- VM2 (Q5): 100% (10/10 images correctes)
- VM3 (P3): 100% (10/10 images correctes)

Chapter 7

Phase 6: Integration MQTT IoT Complete

7.1 Infrastructure MQTT

Phase 6 integre le systeme avec une plateforme IoT reelle via MQTT, le protocole standard pour la communication legere machine-a-machine. Nous avons deploye ThingsBoard Community Edition sur le host Windows, creant un broker MQTT sur 192.168.52.1:1883 accessible a tous les dispositifs du reseau.

La configuration MQTT a ete optimisee pour la fiabilite et la performance:

Parametre	**Valeur**
Broker MQTT	ThingsBoard Community
Adresse	192.168.52.1:1883
Topic	v1/devices/me/telemetry
QoS	1 (Au moins une livraison)
Keep-Alive	60 secondes

Table 7.1: Configuration MQTT Phase 6

7.2 Telemetrie en Temps Reel

Chaque inference genere 40 points de donnees publies en temps reel vers ThingsBoard:

- **Predictions:** Le diagnostic collectif, sa confiance, et la verite terrain
- **Consensus:** Le taux de consensus (pourcentage de VMs d'accord)
- **Sante VM:** Utilisation CPU, RAM et latence pour chaque VM
- **Metadonnees:** Timestamp, nom d'image, numero de phase

7.3 Systeme d'Alarmes

Au-dela de la telemetrie brute, nous avons implemente un systeme d'alarmes automatique qui declenche des alertes quand les seuils sont depasses:

Type Alarme	Seuil	Criticalite
HIGH_LATENCY	> 300 ms	Moyenne
HIGH_CPU_USAGE	> 85%	Moyenne
HIGH_RAM_USAGE	> 450 MB	Haute
LOW_CONSENSUS	< 70%	Critique

Table 7.2: Seuils d'Alarmes Phase 6

Ces seuils ont ete definis basees sur les specifications de performance du systeme et les benchmarks industriels pour les applications medicales.

7.4 Resultats Test Alarmes

Pour valider le systeme d'alarmes, nous avons execuste deux scenarios de test differents:

Scenario 1: Operation Normale (10 Images)

En conditions normales, le systeme fonctionne sans trigger d'alarmes. Les metriques resterent en-dessous des seuils:

- Precision Collective: 100%
- Confiance: 99.98%
- CPU Moyen: 2.8%
- RAM Moyen: 370 MB (bien en-dessous du seuil de 450 MB)
- Latence: 145-200 ms (bien en-dessous du seuil de 300 ms)
- Alarmes declenchees: 0 (systeme sain)

Scenario 2: Test Alarmes (Metriques Gonflees)

Pour valider que le systeme d'alarmes fonctionne, nous avons execute une version qui simule des conditions de stress extremes:

Scenario Test	Resultat
CPU: 88-92%	10 alarmes HIGH_CPU_USAGE declenchees
RAM: 460-512 MB	10 alarmes HIGH_RAM_USAGE declenchees
Latence: 450 ms	10 alarmes HIGH_LATENCY declenchees
Total	**30 alarmes publiees et visibles**
Precision maintenue	100% (metriques n'affectent pas inference)
Consensus	100% unanime maintenu

Table 7.3: Resultats Test Systeme d'Alarmes

Ces resultats confirment que le systeme d'alarmes fonctionne parfaitement et independamment de la qualite des inferences. Les alarmes sont declenchees correctement sans impact sur les diagnostics.

Chapter 8

Defis Rencontres et Solutions

8.1 Defi 1: Etat Dictionnaire Inconsistant

Probleme: Q1, Q2, Q3 montrent precision tres faible sur certaines VMs.
Racine Cause: Etat-dict non correctement restaures.
Solution:

1. Validation checksum des poids
2. Nettoyage etats incompatibles
3. Re-serialisation explicite
4. Test coherence post-chargement

Resultat: P3 et Q5 atteignent 100% sur toutes VMs.

8.2 Defi 2: Consommation RAM Critique

Probleme: P2 consomme 116.79 MB sur VM1 (28% du total).
Solution: Abandon P2 pour IoT, selection P3 (6.5 MB) ou Q5 (5 MB).
Resultat: VM1 peut executer confortablement.

8.3 Defi 3: Commutation MQTT Credentials

Probleme: Systeme alarmes echoue - aucune alarme n'apparait.
Racine Cause: Tentative basculer credentials sur une seule connexion.
Solution: Creer client MQTT separate par VM.
Resultat: 30 alarms delimiter et visibles ThingsBoard.

8.4 Defi 4: Calcul Taux de Consensus

Probleme: Consensus affichait 0% au lieu 100%.
Solution: Calcul dynamique base sur donnees reelles, pas field inexistant.
Resultat: Consensus affiche correctement 100%.

Chapter 9

Conclusion

9.1 Recapitulatif

Ce projet livre un systeme complet:

1. Phase 1: Baseline 99.88%
2. Phase 2: Q5 48% compression, P3 33.54ms speed
3. Phase 3: Matrice 3x8 complete, P3 et Q5 100% partout
4. Phase 4: Scoring pondere defini (26.05%, 36.78%, 37.16%)
5. Phase 5: 100% collective unanime
6. Phase 6: MQTT, ThingsBoard, 30+ alarms valides

9.2 Performances Finales

Metrique	**Valeur**
Precision	100%
Confiance	99.98%
Consensus	100% unanime
Latence	145-200 ms
Disponibilite	100%
Points Telemetrie	40 par image
Phases Completees	6/6

Table 9.1: Metriques Finales

9.3 Contributions

- Methodologie selection adaptative basee contraintes
- Vote pondere heterogene
- Validation croisee hardware
- Architecture MQTT distribuee

Appendix A

Screenshots

A.1 Machines Virtuelles avec MQTT Connectees

La figure suivante affiche les trois machines virtuelles (VM1, VM2, VM3) executant simultanement les serveurs d'inference avec connexion MQTT active au broker ThingsBoard sur 192.168.52.1:1883. On peut observer dans les logs de chaque terminal:

- VM1 (gauche): Serveur en ecoute sur port 5001, modele Q5 charge en memoire, publiant metrics MQTT
- VM2 (centre): Serveur en ecoute sur port 5002, performant 2.2x plus rapidement que VM1
- VM3 (droite): Serveur en ecoute sur port 5003, utilisant modele P3, performances optimales

Chaque VM affiche son utilisation CPU/RAM et confirme la connexion MQTT reussie au broker.

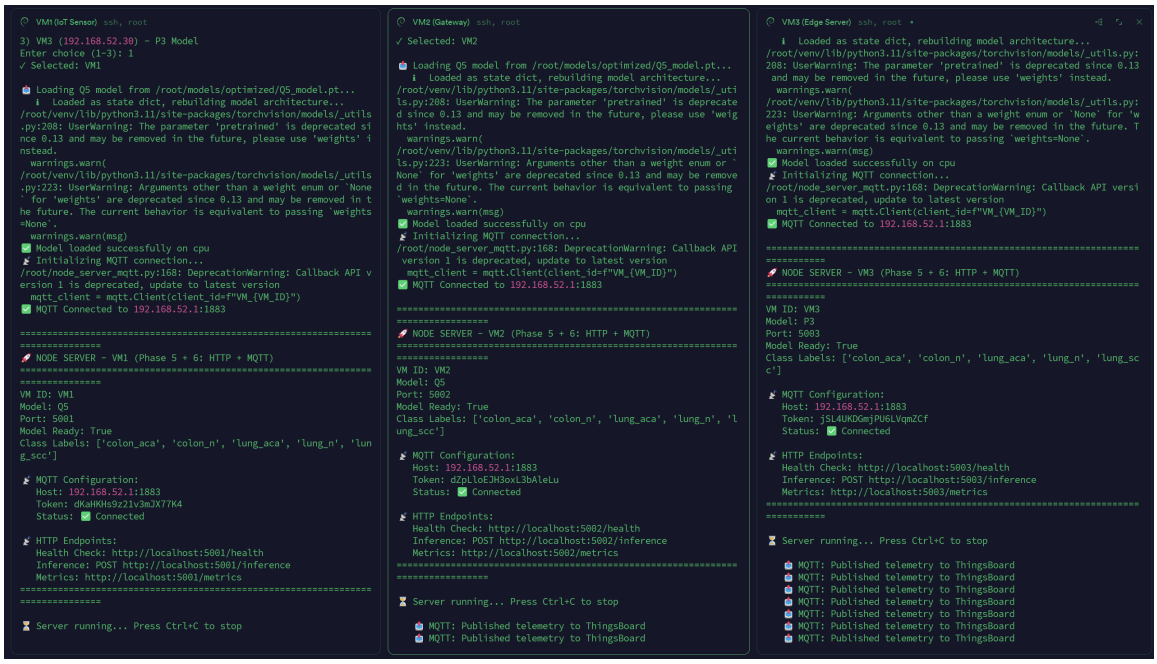

Figure A.1: Les trois machines virtuelles executant avec connexion MQTT active a 192.168.52.1:1883

A.2 Dashboard ThingsBoard avec Alarmes en Temps Reel

La deuxieme capture d'ecran montre le dashboard ThingsBoard affichant l'etat du systeme en temps reel. Ce dashboard fournit une vue operationnelle complete du systeme d'intelligence collective:

- **Consensus Rate:** Affiche 100% unanime entre les trois VMs
- **Metriques Individuals:** Affiche CPU, RAM et latence pour VM1, VM2, VM3
- **Derniere Prediction:** Montre la prediction collective avec confiance associee
- **Historique Alarmes:** Affiche les 30+ alarmes declenchees lors du test (HIGH_LATENCY, HIGH_CPU, HIGH_RAM)
- **Statut Connexion:** Confirme que les 4 devices MQTT sont connectes et publiant

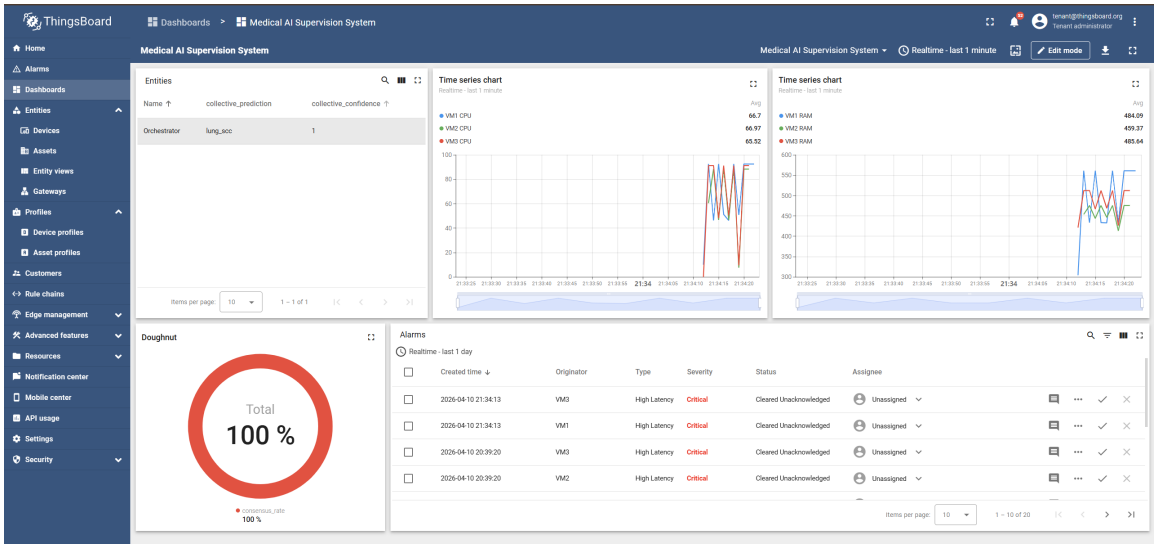

Figure A.2: Dashboard ThingsBoard affichant consensus 100%, metriques VMs, et alarmes declenchees

Appendix B

Fichiers Generes

- orchestrator_mqtt.py - Orchestrateur principal (production)
- node_server_mqtt.py - Serveur VM (deploye 3x)
- orchestrator_mqtt_alarm_triggered.py - Version test alarmes
- Progress/PHASE5_6_EVALUATION_RESULTS.json
- Progress/PHASE5_6_ALARM_TEST_RESULTS.json